

Rome EVM Private - Diff Review

Rome

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

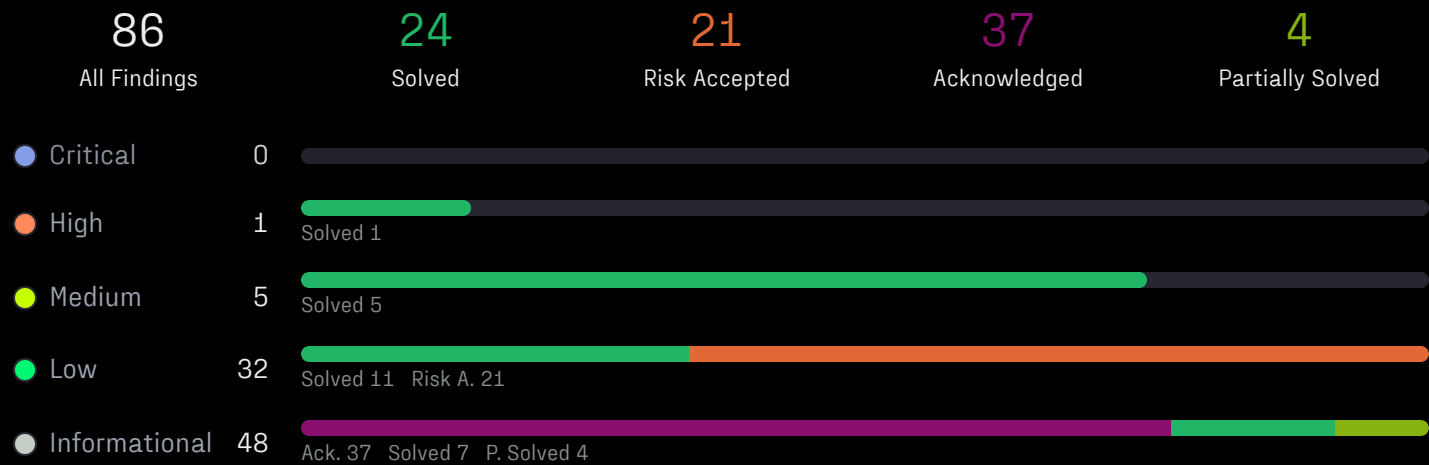
ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Aug 21, 2026 - Sep 5, 2026

SOURCE CODE
evm
rome-evm-private
/solidity/single_state/access_control.sol
/program/src/accounts/account_state.rs
/emulator/src/api/alloc_resource.rs
ASSESSED COMMIT ID
bd199cc
8ceb53a

SEVERITY BREAKDOWN



INTRODUCTION

Rome engaged Halborn to perform a security assessment of Rome EVM, starting on August 21st, 2026 and ending on September 2nd, 2026. The engagement was scoped as a diff audit, so the review covers the code changes between the base and head commits agreed for each repository rather than the full codebase. Commit hashes and the file-level scope are available in the Scope section of this report.

Two repositories were in scope. rome-evm-private holds the on-chain Solana program and the off-chain emulator that mirrors it. evm holds Rome's fork of SputnikVM, which provides the interpreter both of them run. The reviewed checkouts were b7d6421a for rome-evm-private and bd199cc1 for evm.

Rome EVM is the execution layer of Rome Protocol. It runs Ethereum smart contracts on Solana by keeping EVM accounts, storage and contract code in program-derived addresses. A transaction lands through one of two execution models: an atomic path that completes inside a single Solana transaction, and an iterative path that carries VM state across several transactions in a holder account while holding locks on the accounts it touches.

A set of precompiles bridges EVM calldata to Solana. They cover cross-program invocation, SPL token transfers, associated token account creation, PDA derivation and gas-token withdrawal. Each mutating method signs as a program-derived address belonging to the calling EVM address, which is what gives an EVM contract authority over Solana-side assets. A second, cached track stages the same Solana effects in an overlay carried on the journal and submits them at the end of the transaction, which lets SPL flows run on the iterative path.

The emulator answers `eth_call` and gas estimation off-chain. It shares the instruction dispatch and the EVM logic with the program and differs only in where it reads state from, which is what keeps simulation aligned with execution.

Per the statement of work, the engagement was split by file into two concurrent workstreams. Workstream A covered the twelve priority files through manual line-by-line review by a senior security engineer, cross-referenced against output from Halborn's NEO platform. Workstream B covered the remaining files in scope through NEO analysis followed by engineer triage, without independent line-by-line review of the underlying source. Findings from the two workstreams carry different levels of assurance and are distinguished in the findings section.

ASSESSMENT SUMMARY

Halborn was allocated 9 days for this engagement and assigned 1 full-time security engineer to review the code in scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the code in scope.
- Verify that the functionality operates as intended.

The final report contains 86 findings: 1 High, 5 Medium, 32 Low and 48 Informational. Most of the Low and Informational entries are hardening items, divergences from Ethereum semantics that no deployed

contract is currently known to depend on, or defects that sit behind a disabled code path. The High and Medium findings group into three areas: the authority model behind the Solana-signing precompiles, the interpreter fork's handling of attacker-chosen operands, and the accounting and locking behaviour of the iterative execution path.

Every finding was verified against the source a second time before delivery, and executable proofs of concept accompany the findings where the behaviour can be reproduced without a validator.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which should be addressed by the **Rome** team. The main recommendations are:

- Bind delegated precompile calls to an explicit authorization instead of trusting `context.caller` alone, so a contract reached through `DELEGATECALL` cannot sign as the `external_auth` PDA of the user who called it. Apply the check at the single dispatch point in `handler_non_evm.rs` rather than per handler, so every sign-as-caller site on both precompile tracks is covered.
- Keep frame-local failures inside the frame. An unknown opcode, an out-of-range operand or a call to the disabled `MODEXP` address in a callee currently produces `ExitFatal`, which ends the caller's whole transaction instead of returning zero to the `CALL` as Ethereum does.
- Range-check operands before narrowing them. `BYTE` and `SIGNEXTEND` call `as_usize()` on an attacker-chosen word before the bound check, and `CALL` pops `out_offset` and `out_len` only after dispatch; each panics the interpreter, which aborts the Solana instruction rather than reverting the frame.
- Derive each sub-call's budget from the caller's gas argument. The argument is parsed and then discarded, so `transfer()` and `send()` hand the recipient the full remaining budget instead of the 2300 stipend that many deployed contracts rely on as a reentrancy barrier.
- Bound what an iterative session may lock and tie the per-leg fee to the work the leg performed, so a cheap transaction cannot hold locks on accounts it never touches or pad its own leg count against the sender's signed gas limit.
- Enforce the EVM call-depth limit before pushing a frame, and align `SDIV` with Ethereum for `int256::MIN` divided by 1, so recursion is bounded by the specification rather than by host resource exhaustion and signed division does not silently return zero.

Remediation review: The submitted fixes were checked through

`b637afc4110284f9bd3257d37a882e6b7b61a5ac` in `rome-evm-private`, with `evm` pinned to

894180663e23a15de5566d228c7c5d1900f92f35 .

REMEDIATION NOTES

Halborn reviewed the submitted remediations across both `rome-evm-private` and `evm`, through commits `b637afc4110284f9bd3257d37a882e6b7b61a5ac` and `894180663e23a15de5566d228c7c5d1900f92f35`, respectively. All High and Medium severity findings were classified as remediated following this review. These verified improvements strengthen the security of the assessed components and provide an independent basis for confidence in the reviewed implementation.

DEPLOYED PROGRAM VERIFICATION

On September 29, 2026, Halborn independently rebuilt `rome-evm-private` from commit `5e0548798b50bb8f88d994aefc0504462020defe` with the pinned toolchain and locked dependencies, and compared the resulting binary with the deployed Solana mainnet program retrieved through `https://api.mainnet-beta.solana.com` at finalized snapshot slot `451579910` (deployment slot `449766730`). This build revision changes only CI and verification tooling relative to the remediation-reviewed commit `b637afc4110284f9bd3257d37a882e6b7b61a5ac`.

- Program ID: `RomePq9X3iAoTr813HR7uRarafDUJty6GdiEceYpzX`
- Deployed program SHA-256 before metadata masking:
`665c967bd9cb61f809dbf97410a94d355ab674f7bc4d9083daadff7da48cf8db`
- SHA-256 after masking the compilation timestamp and Git revision (deployed and independently rebuilt binaries): `720b61f436ab22db3185e600e930323ad881fd59d61418726193947af9ed01f2`

The executable code matched byte for byte. The complete program binaries matched after the metadata masking and trailing-padding removal described below.

Masking and hash reproduction procedure

1. Dump the deployed program using `solana program dump` with the mainnet RPC endpoint and finalized commitment.
2. Locate the `<<ROME_BUILD_INFO>>` ... `<<END_ROME_BUILD_INFO>>` blob in each binary. Split the bytes from the opening marker up to the closing marker on newline characters into `p`: marker, version, git, rustc, datetime and features.
3. For each binary separately, take its own compilation timestamp (`p[4]`) and Git revision (`p[2]`). For each nonempty value `f`, replace every occurrence of those bytes anywhere in the file with the

same number of ASCII `0` characters: `d = d.replace(f, b"0" * len(f))`. This is a whole-file `bytes.replace` operation, not an offset-limited write.

4. Calculate the SHA-256 of the original and masked files using `solana-verify get-executable-hash`, and compare the masked hashes of the deployed and rebuilt binaries.

Both hashes above follow the `solana-verify` convention of removing trailing NUL bytes before hashing. The before-masking hash therefore excludes trailing allocation padding. Metadata masking uses ASCII `0` characters, not NUL bytes.

Scope

REPOSITORIES

(a) Repository:  `evm`

(b) Assessed Commit ID: `bd199cc1fc4e57f2dac1a6ae5d772552bf8ed9ee`

(a) Repository:  `rome-evm-private`

(b) Assessed Commit ID: `8ceb53a984179aa6def32fe0231d5b86aa62ba96`

REMEDIATION COMMIT ID:

 <https://github.com/rome-protocol/rome-evm-private/pull/536>

 <https://github.com/rome-protocol/rome-evm-private/pull/535>

 <https://github.com/rome-protocol/rome-evm-private/pull/537>

 <https://github.com/rome-protocol/evm/pull/34>

 <https://github.com/rome-protocol/rome-evm-private/pull/521>

 <https://github.com/rome-protocol/rome-evm-private/pull/541>

 <https://github.com/rome-protocol/rome-evm-private/pull/520>

`b39c2b0`

`b39c2b0df14ed44c0ab9ccd45c47867353e71fcf`

 <https://github.com/rome-protocol/rome-evm-private/pull/540>

`8941806`

 <https://github.com/rome-protocol/rome-evm-private/pull/542>

 <https://github.com/rome-protocol/rome-evm-private/pull/523>

 <https://github.com/rome-protocol/evm/pull/36>

🔗 b1dcde6eb58ee9762226c6382a49477c778eb282

🔗 <https://github.com/rome-protocol/rome-evm-private/pull/522>

🔗 <https://github.com/rome-protocol/rome-evm-private/pull/519>

🔗 b637afc4110284f9bd3257d37a882e6b7b61a5ac

Out-of-Scope: New features/implementations after the remediation commit IDs.

Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Insufficient Authorization for DELEGATECALL to Signing Precompiles	● High	7.9	SOLVED 09/15/2026
HAL-002	Guest-Controlled Operand Errors Abort the Caller's Entire EVM Transaction	● Medium	6.8	SOLVED 09/15/2026
HAL-003	Locking Unrelated Accounts in Iterative Sessions Allows Indefinite Targeted Censorship	● Medium	6.3	SOLVED 09/15/2026
HAL-004	SDIV Returns 0 for int256::MIN Divided by 1, Diverging From Ethereum Arithmetic	● Medium	6.3	SOLVED 09/11/2026
HAL-005	BYTE and SIGNEXTEND Panic on Attacker-Chosen Operands, Aborting the Caller's Whole Transaction	● Medium	6.3	SOLVED 09/11/2026
HAL-006	CALL Pops out_offset and out_len After Dispatch, Turning a Frame-Local Error Into a Host Assertion Panic	● Medium	6.3	SOLVED 09/11/2026
HAL-007	ECADD Padding Buffer Is Sized for Output Instead of Input, Panicking on Short Calldata	● Low	4.2	SOLVED 09/15/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-008	BLOCKHASH Returns a Precomputable Hash of the Block Number, Providing No Entropy	● Low	4.2	RISK ACCEPTED 09/18/2026
HAL-009	CPI Read Shortcuts Clone the Entire Target Account, Aborting On-Chain but Passing Emulation	● Low	3.9	RISK ACCEPTED 09/18/2026
HAL-010	Registration and Treasury Instructions Lack Ordering, Bounds, and Authorization Checks	● Low	3.8	RISK ACCEPTED 09/18/2026
HAL-011	Fee Routing Trusts an Unsigned Instruction Prefix and Leaves Several Costs Unmetered	● Low	3.5	RISK ACCEPTED 09/18/2026
HAL-012	Cached-Track Withdrawal to a Fresh ATA Always Reverts on a Builder and Emulator Mismatch	● Low	3.1	SOLVED 09/16/2026
HAL-013	Failed Iterative Transactions Never Settle Their Per-Leg Fees, Leaving the Operator to Pay	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-014	Iterative Submitter Can Pad the Leg Count to Charge the Sender Up to the Signed Gas Limit	● Low	3.1	SOLVED 09/15/2026
HAL-015	Pre-Funding an Uninitialized ATA Prevents Cached Creation	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-016	Unbounded Staged CPI Work Can Exceed the Single-Transaction Commit Budget	● Low	3.1	RISK ACCEPTED 09/18/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-017	Journal Commit Flushes Logs by Emitting Address, Losing Cross-Contract Event Order	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-018	Journal Page Merge Only Relabels the Child Node, Leaving One Journal Node Per Successful Call	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-019	CPI Verification Lets an EVM Caller Fund a System Transfer From the Operator	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-020	Iterative Sessions That Stall Before Commit Leave Operator-Paid PDA Rent Unrecovered	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-021	An EVM Revert Fails the Atomic Solana Instruction: Neither Path Consumes the Nonce or Settles the Fee	● Low	3.1	RISK ACCEPTED 09/18/2026
HAL-022	Registration Does Not Screen Token-2022 Extensions, Admitting a Permanent Delegate That Can Drain the Gas Pool	● Low	2.8	RISK ACCEPTED 09/18/2026
HAL-023	Iterative Holder Reuse and Cleanup Lack Consistent Session-Lifecycle Checks	● Low	2.8	RISK ACCEPTED 09/18/2026
HAL-024	Unchecked Slicing and Offset Arithmetic in Precompile Dispatchers Panic Instead of Reverting	● Low	2.8	SOLVED 09/16/2026
HAL-025	Separating Mint Creation and Initialization Leaves the Mint Open to Third-Party Initialization	● Low	2.6	RISK ACCEPTED 09/18/2026
HAL-026	High-S Signatures Are Accepted, Allowing	● Low	2.6	SOLVED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
	Transaction-Hash Malleability for One Sender			
HAL-027	Wrapped RLP Envelopes and Ignored Trailing Bytes Produce Unbounded Transaction-Hash Aliases	● Low	2.6	RISK ACCEPTED 09/18/2026
HAL-028	RLP Item-Count Check Reads a Malformed Extra Item as Absent, Admitting Non-Canonical Encodings	● Low	2.6	SOLVED 09/16/2026
HAL-029	CREATE and CREATE2 Discard Constructor Revert Data Instead of Bubbling It Up	● Low	2.5	RISK ACCEPTED 09/18/2026
HAL-030	Zero-Length EXTCODECOPY Still Loads and Clones the Entire Account Body	● Low	2.5	RISK ACCEPTED 09/18/2026
HAL-031	Transaction Parsers Under- Validate Fields and Choose the Signature Scheme by RLP Item Presence	● Low	2.5	SOLVED 09/18/2026
HAL-032	Registration Never Checks freeze_authority, Letting an SPL Issuer Freeze the Gas Pool	● Low	2.3	RISK ACCEPTED 09/18/2026
HAL-033	Unverified Source Hash Lets Any Third Party Poison an Inbound Bridge Settlement Marker	● Low	2.1	RISK ACCEPTED 09/18/2026
HAL-034	BN254 and Blake2f Precompiles Return Success on Invalid Input Instead of Failing the Call	● Low	2.1	SOLVED 09/11/2026
HAL-035	An Abandoned CREATE Allocation Blocks Smaller Deployments at the Same Address	● Low	2.1	SOLVED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-036	Pre-Creating the Gas-Pool ATA Permanently Blocks Chain Registration for That Mint	● Low	2.1	SOLVED 09/15/2026
HAL-037	Failed CREATE Rolls Back the Creator's Nonce Increment, Allowing Address Reuse	● Low	2.1	SOLVED 09/15/2026
HAL-038	Iterative Sessions Reuse the Initial Priority Fee and Can Over-Reimburse Later Transactions	● Low	2.1	RISK ACCEPTED 09/18/2026
HAL-039	external_auth Omits chain_id, Sharing One Signing Authority Across Every Chain	● Informational	1.7	ACKNOWLEDGED 09/16/2026
HAL-040	JUMPI Rejects an Untaken Branch Whose Destination Operand Exceeds usize::MAX	● Informational	1.7	SOLVED 09/15/2026
HAL-041	Iterative Sessions Freeze the Clock at the First Leg and Undercount Execute Iterations After a Holder Resize	● Informational	1.7	ACKNOWLEDGED 09/16/2026
HAL-042	Unkeyed Hasher in Storage- Slot Merging Lets Guest- Chosen Keys Collide Deliberately	● Informational	1.6	ACKNOWLEDGED 09/16/2026
HAL-043	Off-Chain Fee Quotes Omit Several Cost Components That On-Chain Settlement Charges	● Informational	1.6	ACKNOWLEDGED 09/16/2026
HAL-044	Memory Limit Validation Occurs After Allocation	● Informational	1.3	SOLVED 09/16/2026
HAL-045	Uncapped Caller Range in JoinTreasure Emulation	● Informational	1.3	SOLVED 09/15/2026

#	TITLE	SEVERITY	SCORE	STATUS
	Costs One RPC Round Trip Per Index			
HAL-046	Uncapped Caller Range in CreateTreasure Emulation Costs an RPC Fetch and CPI Per Index	● Informational	1.3	SOLVED 09/15/2026
HAL-047	Iterative Gas Estimation Has No Aggregate Execution Budget	● Informational	1.3	PARTIALLY SOLVED 09/15/2026
HAL-048	CPI Instruction Verification Deserializes Guest Data Without a Practical Size Bound	● Informational	1.3	SOLVED 09/15/2026
HAL-049	Host Opcodes and Native Precompile Work Escape the Emulation Step Budget	● Informational	1.3	PARTIALLY SOLVED 09/15/2026
HAL-050	Missing Explicit Call-Depth Bound Leaves Recursive Execution to General Resource Limits	● Informational	1.3	ACKNOWLEDGED 09/16/2026
HAL-051	CPI Track Interlock Latches Before the Guarded Call Takes Effect, and Two Reads Bypass It	● Informational	1.3	PARTIALLY SOLVED 09/15/2026
HAL-052	Cached Precompile Overlay Diverges From Real Solana and SPL Execution Semantics	● Informational	1.3	ACKNOWLEDGED 09/16/2026
HAL-053	Host EVM Compatibility and Bookkeeping Need Explicit Semantics	● Informational	1.0	ACKNOWLEDGED 09/16/2026
HAL-054	A Truncated PUSH Immediate Pads on the Wrong Side (Unobservable in Practice)	● Informational	0.8	ACKNOWLEDGED 09/16/2026
HAL-055	Emulator Account Reads Use an Unpinned RPC	● Informational	0.8	ACKNOWLEDGED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
	Commitment With No Slot Binding			
HAL-056	eth_call Driver Ignores Caller Gas Parameters and Lacks Data, Reuse and Start-State Guards	● Informational	0.8	ACKNOWLEDGED 09/16/2026
HAL-057	Emulator Trust-Boundary and Simulation Semantics Need Documentation	● Informational	0.8	ACKNOWLEDGED 09/16/2026
HAL-058	Emulator Has No Memory or Allocation Ceilings on Several Paths	● Informational	0.6	ACKNOWLEDGED 09/16/2026
HAL-059	Emulator Crashes on Malformed Input Instead of Returning a Typed RPC Error	● Informational	0.6	ACKNOWLEDGED 09/16/2026
HAL-060	Emulator CPI, Account Metadata, and Deallocation Accounting Differ From On-Chain Execution	● Informational	0.6	ACKNOWLEDGED 09/16/2026
HAL-061	Unvalidated Account Casts, Unversioned State, and Comment-Only Guards in the Program	● Informational	0.6	PARTIALLY SOLVED 09/15/2026
HAL-062	EVM Compatibility Policies Require Explicit Documentation	● Informational	0.6	SOLVED 09/15/2026
HAL-063	Epoch-Unaware Transfer-Fee Reading Keeps a Disarmed Token-2022 Mint Blocked on Both Precompile Tracks	● Informational	0.4	ACKNOWLEDGED 09/16/2026
HAL-064	Advertised no_std and with-codec Configurations and Standalone Sub-Crate Builds Do Not Compile	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-065	Checkpoint Schema and Execution Limits Need an	● Informational	0.2	ACKNOWLEDGED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
	Explicit Upgrade Contract			
HAL-066	Hand-Written Serialization Mishandles Short Reads and Non-Binary Serde Formats	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-067	Config Advertises Hardfork Semantics the Crate Does Not Implement	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-068	JUMPDEST Bitmap Coherence Is a Trusted Host Invariant	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-069	Unsafe-Code and Warning Lint Policies Need Consistent Enforcement	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-070	Fast Byte-Order Helpers Depend on Native Layout and Little-Endian Targets	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-071	Runtime Completion Helpers Assume StepLimitReached Is Not a Child Exit	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-072	Unresolvable Resolve Guards Poison the Runtime on Drop, so the Host Must mem::forget Every Trap	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-073	Reference Solidity Build, PDA Authority and Precompile ABI Drift	● Informational	0.2	ACKNOWLEDGED 09/16/2026
HAL-074	Interpreter Dependency Uses a Branch Reference While Cargo.lock Pins the Build	● Informational	0.1	ACKNOWLEDGED 09/16/2026
HAL-075	Reference SPL-Wrapper Examples Lack Input Validation, Solvency Checks, and Access Control	● Informational	0.1	ACKNOWLEDGED 09/16/2026
HAL-076	Emulator Account Layer Uses Panicking Assertions	● Informational	0.1	ACKNOWLEDGED 09/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
	and Omits Solana Runtime Checks			
HAL-077	Emulator Clock and Rent Stubs Use Unsynchronized Globals and Fail Open	● Informational	0.1	ACKNOWLEDGED 09/16/2026
HAL-078	Latent SELFDESTRUCT Journal Rollback Behavior While the Opcode Is Disabled	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-079	CALL Gas Arguments Do Not Bound Sub-Calls Under Rome's Fee Model	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-080	Default std Features Also Enable Optional Serialization Dependencies	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-081	Arithmetic Hardening in Valids::is_valid	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-082	Fast Byte-Order APIs Rely on Buffer-Length Contracts	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-083	Unused Dependencies Increase Build and Maintenance Work	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-084	Public API Items Have No Internal Callers in the Reviewed Repository	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-085	Endianness Mismatch Pins DoTxUnsigned Treasury Selection to Wallet 0	● Informational	—	ACKNOWLEDGED 09/16/2026
HAL-086	Ed25519 Authorization Messages Lack Readable Protocol Context	● Informational	—	SOLVED 09/20/2026

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.